Procesado y análisis de datos con *Python*

Manuel Jesús Montilla Gallardo
Alejandro Roldán Moscoso

29 de abril de 2026

Índice

Introducción **2**

1. Fase 1 **2**

1.1. Objetivo 1: Leer en tiempo real un log generado por una CU 2

1.2. Objetivo 2: Identificar los mensajes con información de volumen de datos transferido en DL a nivel SDAP. 3

1.3. Objetivo 3: Guardar en un DataFrame, en tiempo real, los volúmenes de datos indicados para cada UE 4

1.4. Objetivo 4: agregar cada 1 segundo y guardar en otro DataFrame los volúmenes agregados en cada UE. 5

1.5. Objetivo 5: Representar en tiempo real con Dash la evolución de volúmenes agregados para cada UE 6

1.6. Objetivo 6: Cada 30 segundos guardar en un archivo Parquet la evolución de volúmenes agregados en cada UE 8

1.7. Main 9

2. Fase 2 **9**

2.1. Objetivo: Identificar el PLMN, PCI y RNTI de cada UE y añadir esta información en la representación de Dash. 9

3. Interfaz final **10**

Introducción

En este documento se detalla el diseño e implementación de un sistema basado en *Python* para el procesamiento, análisis y visualización de datos en tiempo real. El proyecto se centra en la monitorización de logs de red generados por una Unidad Central (CU) en un entorno de telecomunicaciones, con el fin de examinar y representar el tráfico de datos.

El sistema está diseñado para operar de manera asíncrona y altamente eficiente, extrayendo información crítica sobre el volumen de datos transferidos en el enlace de bajada (*Downlink*) a nivel del protocolo SDAP para distintos equipos de usuario (UE). A lo largo de las fases documentadas en este informe, se abordan retos técnicos fundamentales, estructurados de la siguiente manera:

- Lectura y extracción de logs en tiempo real.
- Agregación temporal de los volúmenes de tráfico.
- Visualización dinámica y en vivo del tráfico de red.
- Almacenamiento periódico del histórico de datos.

De este modo, se proporciona una solución que captura, transforma y expone la información de red sin saturar los recursos del sistema informático, garantizando un flujo continuo entre la generación del log y su representación analítica final.

1. Fase 1

1.1. Objetivo 1: Leer en tiempo real un log generado por una CU

En primer lugar, se define la función `log_reader_producer`, a la cual se le pase como parámetros el nombre del archivo en la variable `path` y la cola donde se irán guardando los datos que van llegando. Esta función se inicializa de forma asíncrona, permitiendo que se ejecute en paralelo con otras tareas (como la web de Dash) sin bloquear el programa.

Se abre el archivo usando `with open(path, ‘‘r’’, encoding‘‘utf-8") as f:` Al usar with, garantizamos que el archivo se cerrará correctamente incluso si el programa falla. encoding=“utf-8”: es fundamental para que el script no se rompa si el log contiene caracteres especiales o símbolos de red propios de los equipos srsRAN.

A continuación, se implementa un ciclo de sondeo (polling) no bloqueante usando `while not file.exists(): await asyncio.sleep(0.5)`. De esta forma, el programa verifica la existencia del archivo cada 500ms, cediendo el control al bucle de eventos para evitar que la CPU

se sature mientras espera.

La instrucción `f.seek(0, 2)` mueve el puntero de lectura al final del archivo. Sirve para ignorar el historial previo y comenzar a procesar únicamente la información que se escriba en el archivo a partir del instante de ejecución.

Dentro de un bucle `while` se comienzan a leer las líneas con `line = f.readline()`, manteniendo al productor en un estado de escucha permanente. Si no se encuentran líneas nuevas (`if not line:`), el sistema espera 100ms (`await asyncio.sleep(0.1)`) antes de reintentar, para optimizar el consumo de recursos.

Finalmente se envía a la cola compartida usando `await queue.put(record)`. Una vez procesado el dato, se inyecta en el objeto `asyncio.Queue`. Este componente actúa como un búfer FIFO (*First-In, First-Out*), desacoplando la velocidad de lectura del log de la velocidad de procesamiento y agregación de datos posterior.

```
    async def log_reader_producer(path: str, queue: asyncio.Queue):
    file = Path(path)
    print(f"Esperando a que se genere el log: {path}...")

    while not file.exists():
        await asyncio.sleep(0.5)

    with open(path, "r", encoding="utf-8") as f:
        print(f"Log {path} abierto. Leyendo en tiempo real...")
        f.seek(0, 2)  # Ir al final del archivo (tail -f)

        while True:
            line = f.readline()
            if not line:
                await asyncio.sleep(0.1)
                continue
```

1.2. Objetivo 2: Identificar los mensajes con información de volumen de datos transferido en DL a nivel SDAP.

La identificación se realiza mediante un patrón de búsqueda definido (*PATTERN_SDAP*) que en cada línea valida:

- Etiqueta SDAP, discriminando los mensajes de otros protocolos.
- Cadena DL: en la arquitectura del log, esto garantiza que solo estamos contabilizando el tráfico de bajada (Downlink) hacia el usuario, discriminándolo del tráfico de subida

(Uplink).

- En esta búsqueda también se identifica el UE y la longitud del mensaje

```
PATTERN_SDAP = re.compile(
    r'^(?P<timestamp>\d{4}-\d{2}-\d{2}T\d{2}:\d{2}:\d{2}\.\d{6})\s+\[SDAP\s*\]\s+\[.\]\s+ue=(?P<ue_id>\d+).*DL:.*pdu_len=(?P<dl_vol>\d+)'
)
```

El módulo re permite trabajar con expresiones regulares. La función re.compile() toma una cadena de texto (el patrón) y la transforma en un objeto de patrón de Python.

Dentro del bucle principal de log_reader_producer, una vez ha abierto el documento con la función *open*, se lee línea a línea y se compara con el patrón establecido. Si hay coincidencia, se guarda los datos de la hora, el UE y la longitud en bytes.

```
match = PATTERN_SDAP.search(line)
if match:
    data = match.groupdict()
    record = {
        "timestamp": datetime.fromisoformat(data["timestamp"]),
        "ue_id": f"UE_{data['ue_id']}",
        "dl_vol": float(data["dl_vol"])
    }
```

1.3. Objetivo 3: Guardar en un DataFrame, en tiempo real, los volúmenes de datos indicados para cada UE

En primer lugar se inicia el cronómetro: `last_agg_time = time.time()` para, dentro de un bucle `while`, recibir los datos de una cola asíncrona. Registra la marca de tiempo inicial en segundos desde la época Unix.

Dentro del bucle, se intenta extraer de la cola con un *timeout* con la línea (`record = await asyncio.wait_for(queue.get(), timeout=0.5)`) Con esto se evita que el consumidor quede bloqueado indefinidamente si no llegan nuevos datos, permitiendo que la ejecución continúe.

Posteriormente, se añade el registro a la cola (`raw_buffer.append(record)`) y se notifica a la cola que el elemento ha sido procesado (`queue.task_done()`). Esto permite que mecanismos de sincronización externa (como `queue.join()`) sepan cuándo el trabajo pendiente ha finalizado.

En el caso en el que expire el *timeout* y no se hayan recibido datos, la instrucción `except asyncio.TimeoutError: pass` permite que el bucle continúe directamente a la evaluación de tiempo, asegurando que, incluso sin datos nuevos, se pueda realizar la agregación si el temporizador ya expiró.

```
async def data_aggregator_consumer(queue: asyncio.Queue):
        global raw_buffer, agg_df

        last_agg_time = time.time()

        while True:
            # Recibir dato de la cola (espera máximo 0.5s)
            try:
                record = await asyncio.wait_for(queue.get(),
   timeout=0.5)
                raw_buffer.append(record)
                queue.task_done()
            except asyncio.TimeoutError:
                pass # No llegaron datos nuevos en este medio
```

1.4. Objetivo 4: agregar cada 1 segundo y guardar en otro DataFrame los volúmenes agregados en cada UE.

En primer lugar se observa que se cumpla un segundo desde el último instante de tiempo guardado, que se comprueba con la condición: `if current_time - last_agg_time >= 1.0`.

Tras ello, se realiza una normalización temporal mediante una técnica de truncado. Utilizando la librería de alto rendimiento Polars, se eliminan los milisegundos de las etiquetas de tiempo de cada registro, de modo que todos los eventos ocurridos dentro del mismo segundo virtual reciben una marca de tiempo idéntica.

```
# Truncar los timestamps al segundo para agrupar
df_raw = df_raw.with_columns(
    pl.col("timestamp").dt.truncate("1s").alias("time_window"))
```

Posteriormente, se ejecuta una operación de agrupación multidimensional (group_by). El sistema segmenta los datos basándose en dos criterios simultáneos: la ventana temporal y el identificador del equipo de usuario (UE). Dentro de cada grupo creado, se aplica una función de suma sobre los bytes capturados, obteniendo así el volumen total transferido por cada usuario durante ese segundo específico.

```
# Truncar los timestamps al segundo para agrupar
```

```
# Agrupar por ventana de 1 segundo y por UE, sumar el volumen
df_grouped = (
    df_raw.group_by(["time_window", "ue_id"])
    .agg(pl.col("dl_vol").sum().alias("dl_vol_sum"))
)
```

Finalmente, el resultado de esta operación se integra en un DataFrame histórico maestro mediante concatenación vertical. Una vez que los datos han sido consolidados en esta estructura global, el buffer de datos crudos se libera por completo. Esta gestión del ciclo de vida de los datos garantiza la estabilidad del sistema a largo plazo, manteniendo un consumo de memoria RAM reducido y constante, independientemente de la duración de la simulación o del volumen total de tráfico procesado.

```
# Unir al DataFrame global agregado
agg_df = pl.concat([agg_df, df_grouped], how="vertical")

# Limpiar el buffer crudo para el siguiente segundo
raw_buffer = []
```

1.5. Objetivo 5: Representar en tiempo real con Dash la evolución de volúmenes agregados para cada UE

Para representar en tiempo real, en primer lugar se crea el contenedor raíz `html.Div([...])`. Dentro de ella se encuentran:

- La etiqueta HTML `html.H3(‘‘Evolución de Volúmenes DL SDAP por UE")`.
- El contenedor gráfico `dcc.Graph(id="live-graph")`, que establede una identidad única que el sistema de *callbacks* utilizará para enviar los datos calculados desde Python hacia el navegador.
- `dcc.Interval(id=ïnterval-component", interval=1000, n_intervals=0)` Este componente es la clave del tiempo real.
    - `interval=1000`: Define el ciclo de disparo en milisegundos.
    - `n_intervals=0`: Es el contador interno que Dash incrementa cada vez que el tiempo se cumple. Este valor es la *Input* que dispara la función `update_graph`, obligando a la interfaz a refrescarse constantemente.

```
app = Dash(__name__)

app.layout = html.Div([
    html.H3("Evolución de Volúmenes DL SDAP por UE"),
```

```
    dcc.Graph(id="live-graph"),
    dcc.Interval(id="interval-component", interval=1000,
   n_intervals=0) # Actualizar cada 1s
])
```

Dentro del *callback*, se definen el *input* y el *output*:

- `Output("live-graph", "figure")`
  Define el destino de la información. Le dice a Dash que lo que esta función devuelva, debe actualizarse en la gráfica.
- `Input(´´interval-component'', ´´n_intervals'')`
  Define el disparador. Le dice a Dash que ejecute esta función cada vez que el componente `interval-component` cambie su valor `n_intervals`. Como este componente suma 1 cada segundo, el gráfico se refresca automáticamente cada segundo.

```
@app.callback(
    Output("live-graph", "figure"),
    Input("interval-component", "n_intervals")
)
```

A continuación, se define la función de la gráfica (`def update_graph(n):`), la cual Recibe el parámetro `n` (que es el valor actual del contador de intervalos). En esta función se va a leer la variable `agg_df` definida fuera de esta función. Es necesario para acceder al DataFrame maestro donde se guarda todo el histórico de datos.

Si el script acaba de arrancar y aún no ha llegado ningún mensaje SDAP al log, devuelve un gráfico vacío con un título informativo en lugar de intentar procesar datos que no existen (evitando errores). Esto se define dentro de la condición `if agg_df.is_empty()`.

Para asegurar que las filas del DataFrame estén ordenadas cronológicamente antes de pasárselas a Plotly se usa `sorted_df = agg_df.sort("time_window")`. Sin esto, Plotly uniría los puntos en el orden en que llegaron.

El último paso antes de graficar es convertir el DataFrame de *Polars* (optimizado para cálculos rápidos) a un DataFrame de *Pandas* (optimizado para visualización y manipulación de series temporales con *Resampling*). Para ello se usa `df_pd = sorted_df.to_pandas()`.

```
def update_graph(n):
    global agg_df
    if agg_df.is_empty():
        return px.line(title="Esperando datos...")

    # Ordenar estrictamente el DataFrame de Polars por tiempo
    sorted_df = agg_df.sort("time_window")
```

```
    # ----------------------------

    # Convertir a Pandas el DataFrame ya ordenado
    df_pd = sorted_df.to_pandas()
```

Finalmente se crea el gráfico.

```
    # Crear gráfico de líneas
    fig = px.line(
        df_pd,
        x="time_window",
        y="dl_vol_sum",
        color="ue_id",
        markers=True,
        title="Tráfico DL Agregado (1s) por User Equipment (Corregido)",
        labels={"time_window": "Tiempo", "dl_vol_sum": "Volumen Total (Bytes)"}
    )
    return fig
```

1.6. Objetivo 6: Cada 30 segundos guardar en un archivo Parquet la evolución de volúmenes agregados en cada UE

```
async def parquet_saver():
    global agg_df
    while True:
        await asyncio.sleep(30) # Esperar 30 segundos
        if not agg_df.is_empty():
            print(f"[{datetime.now().strftime('%H:%M:%S')}] Guardando evolución en {PARQUET_FILE}...")
            # Guardar sobrescribiendo el histórico
            agg_df.write_parquet(PARQUET_FILE)
            print("Guardado exitoso.")
```

El flujo de trabajo consiste en esperar un intervalo de 30 segundos, verificar si el DataFrame contiene datos nuevos y, en caso afirmativo, realizar una operación de escritura. En esta implementación, se sobrescribe el archivo con la evolución completa acumulada hasta el momento, asegurando que el archivo final contenga toda la serie temporal desde el inicio de la captura.

Para ello, se define una función asíncrona `async def parquet_saver():` que se ejecuta de forma concurrente. Esto permite que el guardado ocurra en segundo plano sin pausar la inges-

ta de logs ni la actualización de la interfaz Dash. Se establece el tiempo de guardado de 30 segundos mediante `await asyncio.sleep(30)` dentro de un bucle `while`. Si el DataFrame maestro está vacío (`if not agg_df.is_empty():`) el sistema omite la operación de escritura para evitar generar archivos redundantes o corruptos.

Finalmente, se almacenan los datos (`agg_df.write_parquet(PARQUET_FILE)`) usando el formato Apache Parquet. A diferencia de un CSV, Parquet guarda los datos en columnas, más eficiente a la hora de almacenar ya que permite una comprensión mejor y un acceso más rápido a los datos.

1.7. Main

La función main gestiona las tareas independientes dentro de un único hilo de control principal.

- `asyncio._task()` Cada llamada registra una corrutina en el bucle de eventos (*event loop*) de Python. Esto significa que el Productor, el Consumidor y el Guardador de Parquet empiezan a ejecutarse en paralelo. Python alterna entre ellos cuando uno de ellos alcanza un punto de espera (await), garantizando eficiencia máxima.
- `await asyncio.to_thread(app.run, ...)`
  Esta es una instrucción crítica. La función ‘app.run‘ de Dash es una operación bloqueante (se queda esperando conexiones web indefinidamente). Si la ejecutáramos como una corrutina normal, el bucle de eventos se detendría ahí y el resto de tareas (leer el log, procesar datos) morirían. ‘to_thread‘ envía a ‘app.run‘ a un hilo separado del sistema operativo, permitiendo que la interfaz web funcione en segundo plano mientras el bucle principal de `asyncio` sigue gestionando el flujo de datos.
- `if __name__ == '__main__': asyncio.run(main())`
  Es la protección estándar de Python. Garantiza que el script solo inicie la ejecución si se invoca directamente desde la terminal, evitando problemas si alguna vez decides importar partes de este código en otro script. `asyncio.run(main())` es el encargado de arrancar el *Event Loop*, el cual gestionará el ciclo de vida de todas las tareas creadas.

2. Fase 2

2.1. Objetivo: Identificar el PLMN, PCI y RNTI de cada UE y añadir esta información en la representación de Dash.

Primeramente se define un patrón de búsqueda independiente (PATTERN_CONTEXT) enfocado en capturar los eventos de creación o configuración del UE:

```
PATTERN_CONTEXT = re.compile(
    r'ue=(?P<ue_id>\d+).*?plmn=(?P<plmn>\d+).*?pci=(?P<pci>\d+).*?
   rnti=(?P<rnti>0x[0-9a-fA-F]+)'
)
```

En segundo lugar, como los datos en el log vienen separados, se introduce una estructura de datos tipo diccionario denominada ue_metadata. Cuando se detecta una línea con configuración de red, actualiza los valores en el diccionario asociados a ese identificador de usuario (ue_id). Posteriormente, cuando el productor lee una línea de tráfico SDAP, consulta este diccionario para "pegar" o inyectar los metadatos de red más recientes al registro de volumen antes de enviarlo a la cola de procesamiento. En caso de que un UE comience a transmitir antes de que se lea su configuración, el sistema asigna el valor seguro "N/A".

En tercer lugar, el esquema del DataFrame histórico (agg_df) se amplía respecto a la fase uno, para soportar estas nuevas columnas de texto. Durante la fase de agregación por ventanas de 1 segundo, la instrucción lógica para consolidar estos datos fue:

```
pl.col("pci").last(), pl.col("plmn").last(), pl.col("rnti").last())
```

Por último, para la capa de visualización, se procesó el DataFrame justo antes de renderizar la gráfica. En lugar de utilizar únicamente el ue_id como separador de color, se generó dinámicamente una nueva columna llamada UE_Info concatenando toda la información extraída:

```
df_pd["UE_Info"] = (
    df_pd["ue_id"] +
    " (RNTI: " + df_pd["rnti"] +
    ", PCI: " + df_pd["pci"] +
    ", PLMN: " + df_pd["plmn"] + ")"
)
```

3. Interfaz final

La aplicación se despliega sobre un servidor web local que escucha en una dirección IP y un puerto específicos (por defecto http://127.0.0.1:8050). Al acceder a esta URL, el usuario establece una conexión mediante protocolo HTTP que sirve los componentes visuales de la interfaz, los cuales se actualizan automáticamente cada segundo sin necesidad de recargar la página gracias al componente dcc.Interval.

Eje de Ordenadas (Bytes): Representa el volumen de datos en bajada (Downlink) acumulado por cada segundo. La escala se ajusta automáticamente, mostrando prefijos como "0.8M"para indicar Megabytes si el tráfico es elevado.

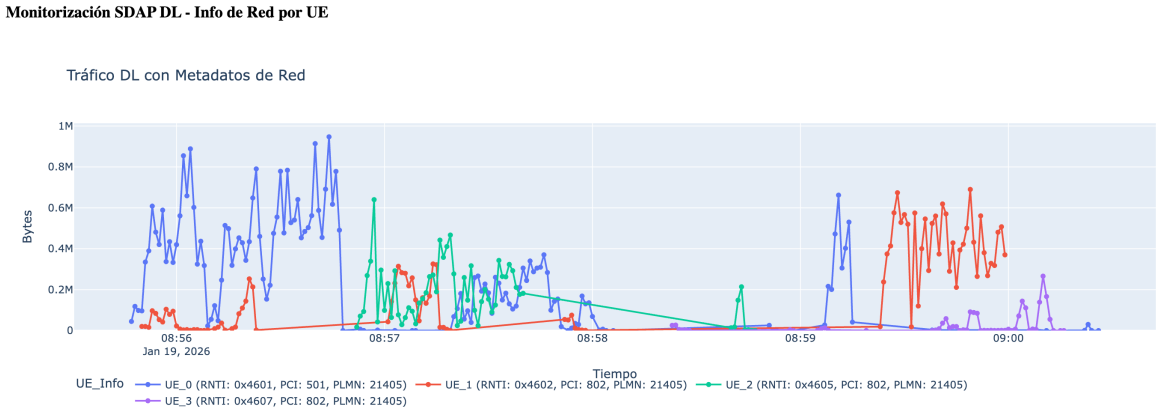

Figura 1: Interfaz de monitorización en tiempo real con Dash mostrando el tráfico DL y metadatos de red por UE.

Eje de Abscisas (Tiempo): Muestra la evolución temporal con precisión de segundos. Como se observa en la captura, la gráfica es capaz de representar periodos de actividad y periodos de silencio (caídas a cero), proporcionando una visión fiel del ciclo de vida de la conexión de cada UE.

Leyenda Informativa Dinámica: Este es uno de los puntos más relevantes del diseño. Cada línea de color representa un equipo de usuario distinto. La leyenda no solo indica el ID del terminal (UE_0, UE_1, etc.), sino que inyecta en tiempo real el contexto de red capturado del módulo [CU-UEMNG]:

- RNTI: Identificador temporal en la celda (ej. 0x4601).
- PCI: Identificador de la celda física (ej. 501 u 802).
- PLMN: Código de la red del operador (ej. 21405).